

```

from pyspark.sql import SparkSession
import pyspark.sql.functions as F
from pyspark.sql.types import DoubleType
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.classification import MultilayerPerceptronClassifier
from pyspark.ml.tuning import ParamGridBuilder, CrossValidator
from pyspark.ml.evaluation import BinaryClassificationEvaluator

print("1. Εκκίνηση αυτόνομης SparkSession για Neural Networks (Διορθωμένο)...")
spark = SparkSession.builder \
    .appName("MLP_SMOTE_Corrected_GridSearch") \
    .master("local[*]") \
    .config("spark.driver.memory", "8g") \
    .getOrCreate()

print("2. Φόρτωση Δεδομένων (Απευθείας από το ΔΙΟΡΘΩΜΕΝΟ SMOTE Gold dataset)...")
train_gold = spark.read.parquet("../data/train_gold_corrected.parquet")
test_gold = spark.read.parquet("../data/test_gold_corrected.parquet")

train_gold = train_gold.withColumn("stroke",
train_gold["stroke"].cast(DoubleType()))
test_gold = test_gold.withColumn("stroke",
test_gold["stroke"].cast(DoubleType()))
train_gold = train_gold.withColumn("cluster",
F.col("cluster").cast(DoubleType()))
test_gold = test_gold.withColumn("cluster", F.col("cluster").cast(DoubleType()))

print("3. Feature Augmentation (Ενσωμάτωση K-Means Cluster)...")
assembler = VectorAssembler(inputCols=["features", "cluster"],
outputCol="augmented_features")

train_augmented =
assembler.transform(train_gold).drop("features").withColumnRenamed("augmented_features",
"features")
test_augmented =
assembler.transform(test_gold).drop("features").withColumnRenamed("augmented_features",
"features")

train_augmented.cache()
test_augmented.cache()

print("4. Υπολογισμός Διαστάσεων Εισόδου (Input Size)...")
input_size = len(train_augmented.select("features").first()[0])
print(f" -> Βρέθηκαν {input_size} χαρακτηριστικά εισόδου (Features + Cluster).")

print("5. Ορισμός Neural Network και ΒΕΛΤΙΣΤΟΠΟΙΗΜΕΝΟΥ Πλέγματος Παραμέτρων...")
# Ορίζουμε το mlp_base με σταθερό blockSize για σταθερά mini-batches
mlp_base = MultilayerPerceptronClassifier(featuresCol="features",

```

```

labelCol="stroke", blockSize=128, seed=22390225)

# ΟΙ ΑΛΛΑΓΕΣ:
# - Κρατάμε τις έξυπνες, ρηχές αρχιτεκτονικές σου.
# - maxIter: Δοκιμάζουμε [50, 100] για επαρκή σύγκλιση.
# - stepSize: Το micro-tuning του learning rate εμποδίζει την απότομη αποστήθιση
του SMOTE.
paramGrid = (ParamGridBuilder()
    .addGrid(mlp_base.layers, [
        [input_size, 4, 2],      # Ακραία μινιμαλιστικό (μόνο 4 νευρώνες)
        [input_size, 2]         # Χωρίς κρυφό επίπεδο (Linear / Logistic baseline)
    ])
    .addGrid(mlp_base.maxIter, [50, 100])
    .addGrid(mlp_base.stepSize, [0.01, 0.1]) # Έλεγχος της ταχύτητας
εκπαίδευσης
    .build())

# ΔΙΟΡΘΩΣΗ: Αλλαγή σε areaUnderROC για να μεγιστοποιήσουμε το Recall και τη
σωστή
# διαβάθμιση των πιθανοτήτων, χωρίς να επηρεαζόμαστε από το Precision του SMOTE
evaluator = BinaryClassificationEvaluator(
    labelCol="stroke",
    rawPredictionCol="rawPrediction",
    metricName="areaUnderROC"
)

cv = CrossValidator(estimator=mlp_base,
    estimatorParamMaps=paramGrid,
    evaluator=evaluator,
    numFolds=3,
    seed=22390225)

print("6. Εκτέλεση Cross-Validation στο SMOTE Dataset...")
cv_model = cv.fit(train_augmented)
best_mlp = cv_model.bestModel

print("\n" + "="*60)
print("[ΕΥΡΕΣΗ ΠΑΡΑΜΕΤΡΩΝ NEURAL NETWORK ΟΛΟΚΛΗΡΩΘΗΚΕ]")
print(f" -> Βέλτιστη Αρχιτεκτονική (Layers): {best_mlp._java_obj.getLayers()}")
print(f" -> Βέλτιστος αριθμός Επαναλήψεων (maxIter): {best_mlp._java_obj.getMaxIter()}")
print("="*60 + "\n")

print("7. Παραγωγή προβλέψεων στο άγνωστο Test Set...")
mlp_preds = best_mlp.transform(test_augmented)

print("8. Αποθήκευση των τελικών προβλέψεων...")
output_path = "../data/mlp_predictions.parquet"
mlp_preds.select("stroke", "prediction",

```

```

"probability").write.mode("overwrite").parquet(output_path)

print(f"=====")
print(f"[ΕΠΙΤΥΧΙΑ] Το Διορθωμένο Neural Network εκπαιδεύτηκε και αποθηκεύτηκε!")
print(f"=====")

spark.stop()

```

1. Εκκίνηση αυτόνομης SparkSession για Neural Networks (Διορθωμένο)...

```

WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
26/06/12 03:59:57 WARN Utils: Your hostname, cachyos-x8664, resolves to a
loopback address: 127.0.1.1; using 192.168.1.5 instead (on interface enp4s0)
26/06/12 03:59:57 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another
address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use
setLogLevel(newLevel).
26/06/12 03:59:57 WARN NativeCodeLoader: Unable to load native-hadoop library for
your platform... using builtin-java classes where applicable

```

2. Φόρτωση Δεδομένων (Απευθείας από το ΔΙΟΡΘΩΜΕΝΟ SMOTE Gold dataset)...

3. Feature Augmentation (Ενσωμάτωση K-Means Cluster)...

4. Υπολογισμός Διαστάσεων Εισόδου (Input Size)...

-> Βρέθηκαν 15 χαρακτηριστικά εισόδου (Features + Cluster).

5. Ορισμός Neural Network και ΒΕΛΤΙΣΤΟΠΟΙΗΜΕΝΟΥ Πλέγματος Παραμέτρων...

6. Εκτέλεση Cross-Validation στο SMOTE Dataset...

```

=====
[ΕΥΡΕΣΗ ΠΑΡΑΜΕΤΡΩΝ NEURAL NETWORK ΟΛΟΚΛΗΡΩΘΗΚΕ]
-> Βέλτιστη Αρχιτεκτονική (Layers): [I@4205cc2a
-> Βέλτιστος αριθμός Επαναλήψεων (maxIter): 100
=====

```

7. Παραγωγή προβλέψεων στο άγνωστο Test Set...

8. Αποθήκευση των τελικών προβλέψεων...

```

=====
[ΕΠΙΤΥΧΙΑ] Το Διορθωμένο Neural Network εκπαιδεύτηκε και αποθηκεύτηκε!
=====

```